《控制系统仿真与 CAD》课程设计小报告

选题二：Bode 图法设计串联校正装置

单位负反馈系统串联校正设计与 MATLAB/Simulink 仿真

0.1. 实验目的

掌握利用 Bode 图法设计串联校正装置的基本思路，理解静态速度误差系数、相位裕度和增益裕度对控制系统性能的影响。通过 MATLAB 完成控制器参数计算、频域指标验证和响应曲线绘制，并使用 Simulink 建立单位负反馈仿真结构图。

0.2. 实验设备

本实验使用 MATLAB、Control System Toolbox 和 Simulink。MATLAB 程序用于传递函数建模、Bode 图分析、稳定裕度计算和响应仿真；Simulink 用于搭建校正后单位负反馈系统结构并导出仿真结构图。

0.3. 设计任务

选题二要求对单位负反馈控制系统进行串联校正。被控对象开环传递函数为

$$G_0(s)=\frac{K}{s(s+1)(s+4)}$$

要求采用 Bode 图法设计串联校正装置 $G_{c(s)}$，使校正后系统满足静态速度误差系数 $K_v=10\text{s}^{-1}$，相位裕度不小于 50°，增益裕度不小于 10 dB。

0.4. 实验原理

单位负反馈系统的静态速度误差系数为

$$K_v=\lim_{s\to 0} sG_0(s)=\frac{K}{4}$$

由 $K_v=10\text{s}^{-1}$ 可得 $K=40$。因此未校正对象取为

$$G_0(s)=\frac{40}{s(s+1)(s+4)}$$

未校正系统在较高穿越频率附近相位滞后较大，直接闭环时相位裕度不足。为保持低频增益和 K_v 不变，同时降低穿越频率、提高稳定裕度，本文采用直流增益为 1 的滞后校正环节：

$$G_{c(s)}=\frac{Ts+1}{\beta Ts+1}$$

MATLAB 计算得到 $T=20$ s，$\beta=17.838657$，所以

$$G_{c(s)}=\frac{20s+1}{356.773145s+1}$$

该校正环节的直流增益为 1，因此校正后仍有 $K_v=10\text{s}^{-1}$。

0.5. 实验内容与步骤

1. 根据静态速度误差系数关系 $K_v = \frac{K}{4}$ 确定系统开环增益 $K = 40$。
2. 使用 MATLAB 建立未校正开环传递函数 $G_0(s)$，并计算校正前系统频域特性。
3. 选取目标增益穿越频率约为 0.5 rad/s，按 Bode 图法确定滞后校正装置参数 T 和 β。
4. 构造校正后开环传递函数 $L(s) = G_{c(s)}G_0(s)$，使用 `margin` 验证相位裕度和增益裕度。
5. 建立校正后闭环系统，绘制阶跃响应和斜坡输入跟踪曲线，验证稳定性与稳态误差。
6. 使用 Simulink 建立串联校正单位负反馈结构，导出仿真结构图用于报告说明。

0.6. 仿真结构

校正后的系统采用单位负反馈结构，串联校正装置位于输入误差信号与被控对象之间。

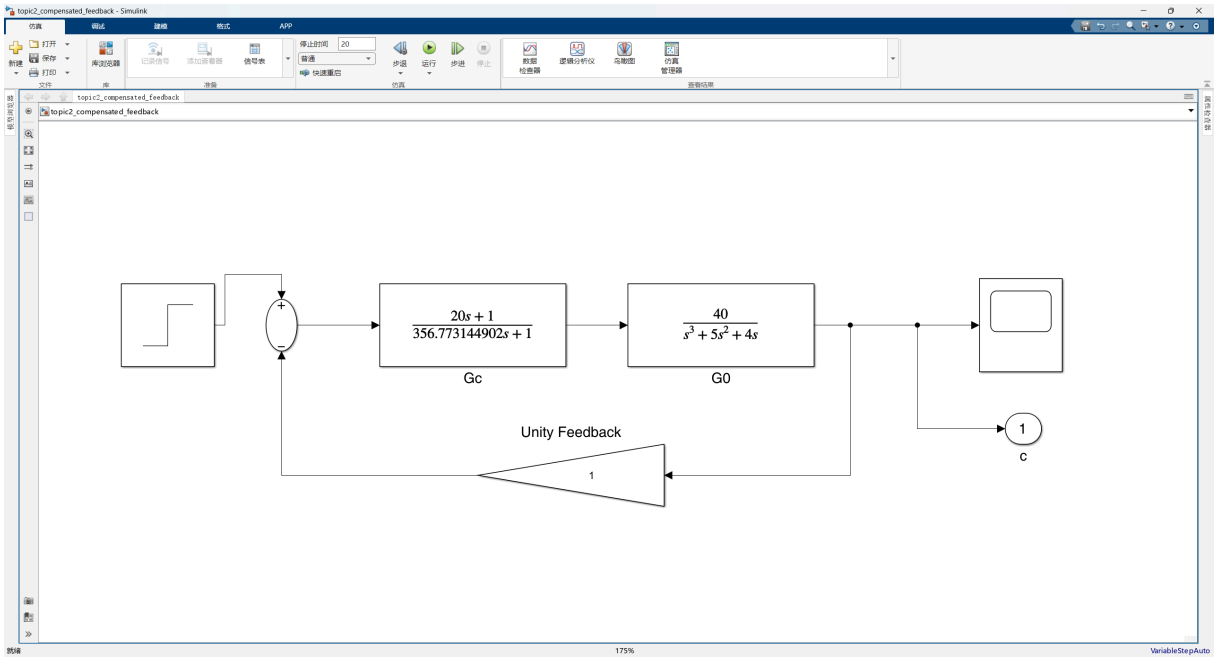


图 1　Simulink 校正系统单位负反馈仿真结构图

0.7. 仿真结果

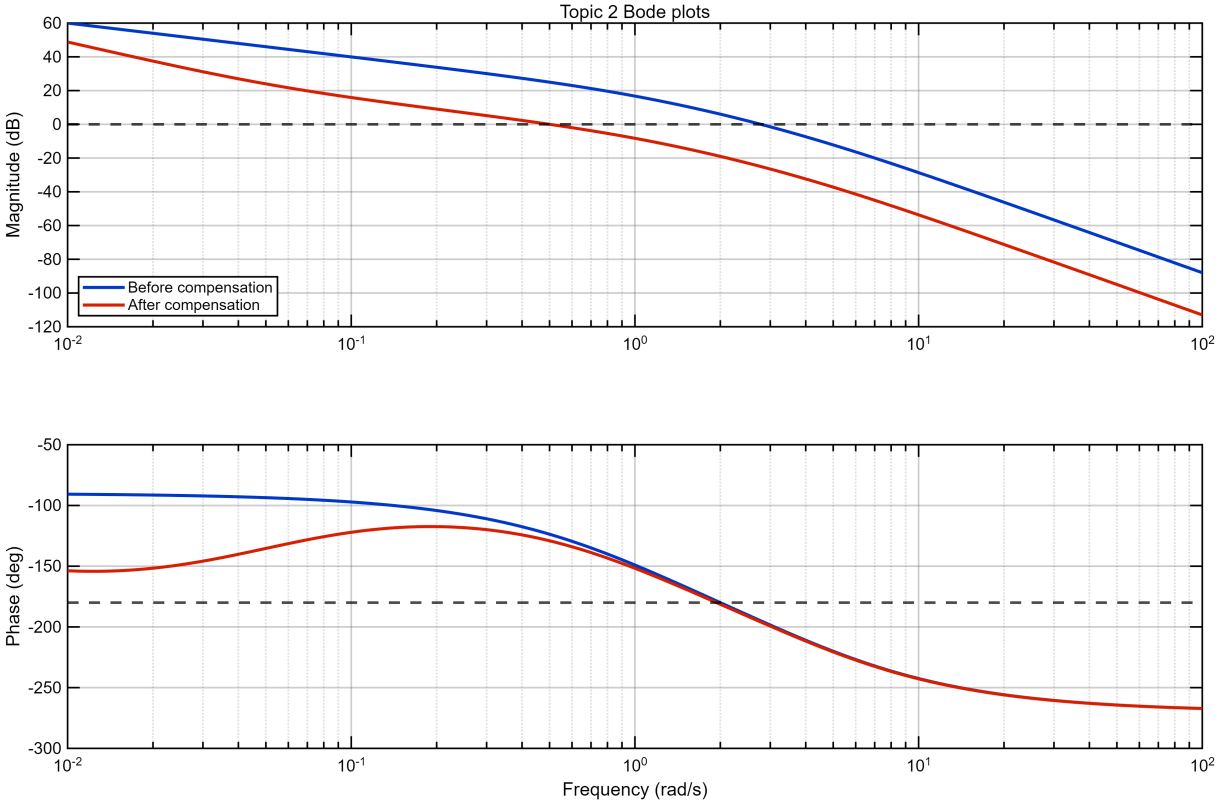


图 2　校正前后开环 Bode 图对比

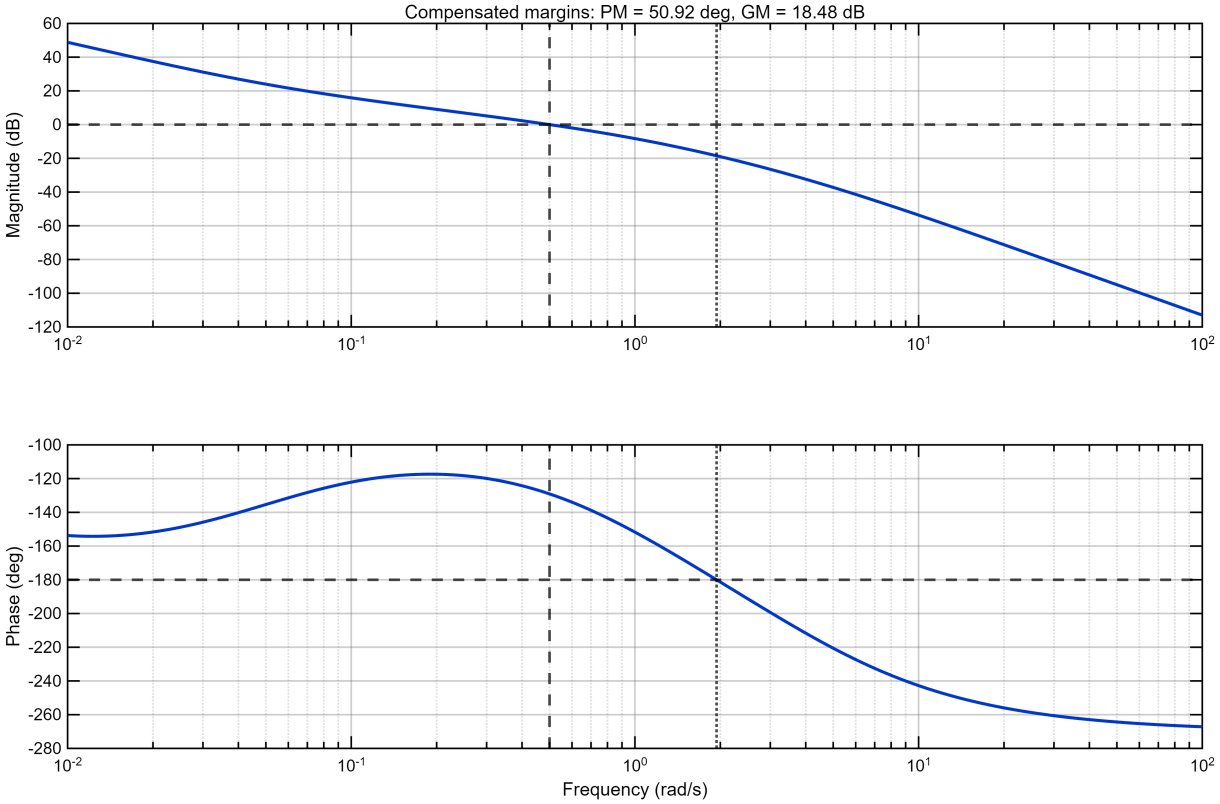


图 3　校正后开环系统稳定裕度

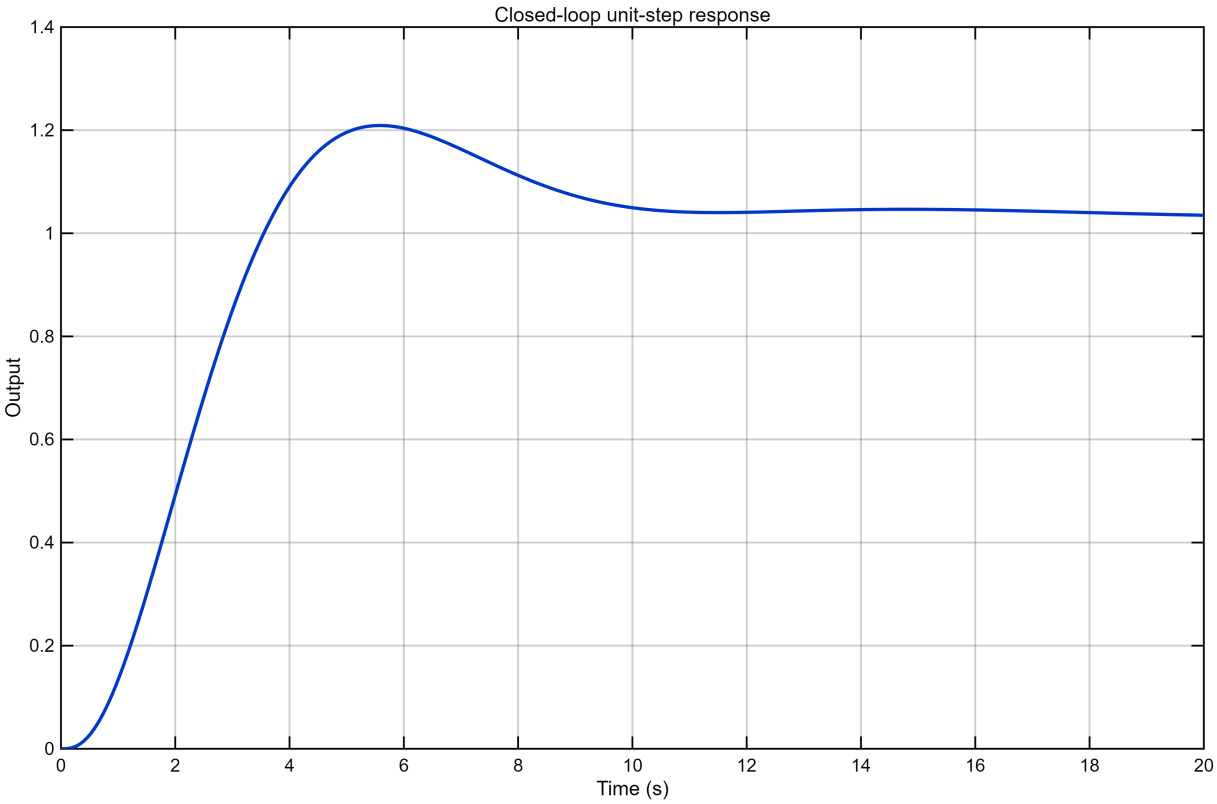


图 4　校正后闭环单位阶跃响应

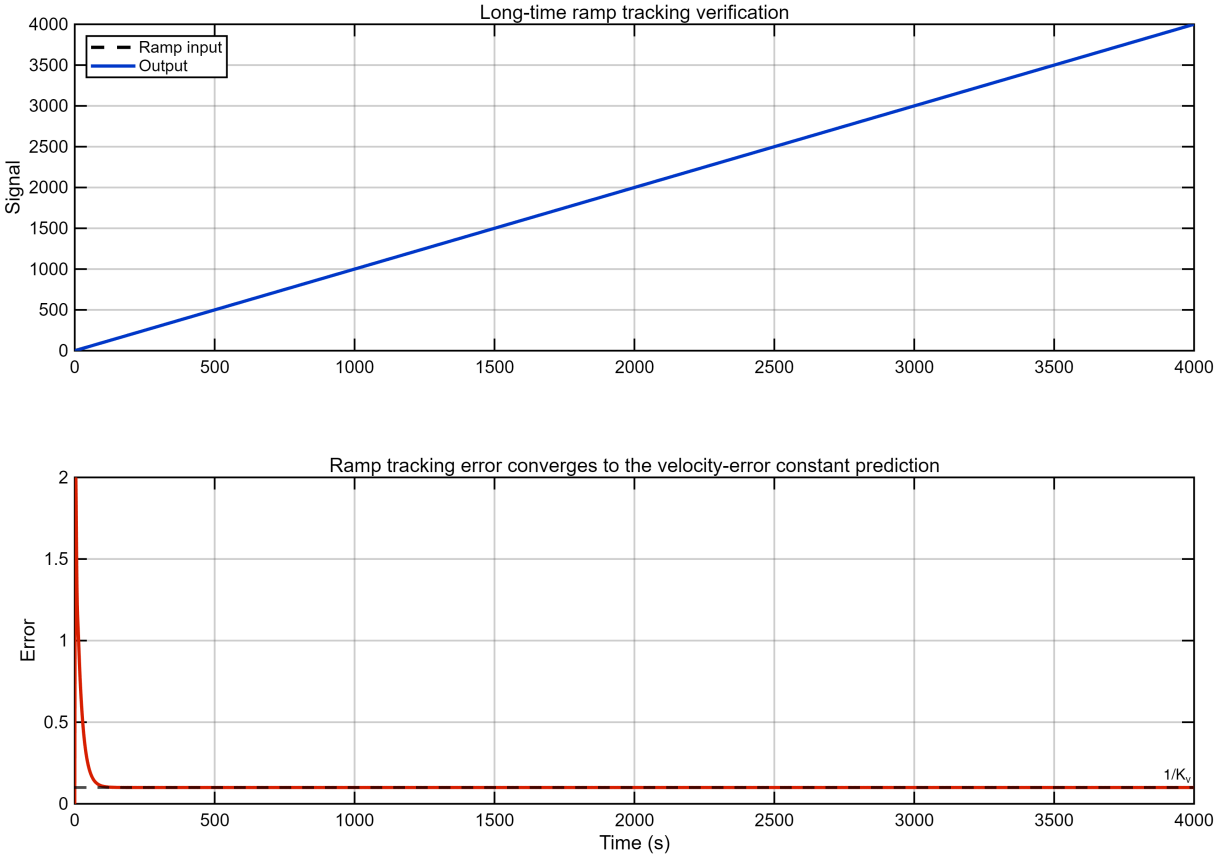


图 5　斜坡输入跟踪与稳态误差验证

指标	设计要求	仿真结果
K_v	$10\,\mathrm{s}^{-1}$	$10.000000\,\mathrm{s}^{-1}$
相位裕度	不小于 50°	50.920661°
增益裕度	不小于 10 dB	18.478168 dB
增益穿越频率	由校正确定	0.499997 rad/s
稳态斜坡误差	$\frac{1}{K_v} = 0.100000$	长时间仿真值 0.100000

表 1　主要性能指标汇总

0.8. 结果分析

从 Bode 图可见，滞后校正后开环增益穿越频率降低到约 0.5 rad/s，系统在增益穿越频率处获得了更大的相位裕度。仿真结果显示相位裕度为 50.920661°，增益裕度为 18.478168 dB，均满足题目要求。由于校正环节的直流增益为 1，低频增益没有改变，静态速度误差系数仍为 $K_v = 10\,\mathrm{s}^{-1}$。

阶跃响应曲线表明校正后闭环系统稳定。由于滞后校正环节的极点较低，斜坡误差的收敛过程较慢，因此程序采用长时间斜坡仿真单独验证稳态误差。仿真曲线显示误差最终收敛到 $\frac{1}{K_v} = 0.1$ 附近，与速度误差系数的理论结果一致。本题的核心频域指标和斜坡稳态误差均已由程序断言验证通过。

0.9. 程序清单

本题使用 `tf` 建立传递函数模型，使用 `margin` 验证相位裕度和增益裕度，使用 `feedback`、`lsim` 等函数绘制响应曲线，并自动更新 Simulink 结构图。完整程序如下。

```
clear; close all; clc;

projectRoot = fileparts(fileparts(mfilename('fullpath')));
figDir = fullfile(projectRoot, 'figures', 'topic2');
buildDir = fullfile(projectRoot, 'build');
if ~exist(figDir, 'dir'), mkdir(figDir); end
if ~exist(buildDir, 'dir'), mkdir(buildDir); end

s = tf('s');
K = 40;
G0 = K / (s * (s + 1) * (s + 4));

Kv = K / 4;
targetWc = 0.5;
Tlag = 20;
plantMagAtWc = abs(squeeze(freqresp(G0, targetWc)));
beta = sqrt((plantMagAtWc^2 * (1 + (targetWc * Tlag)^2) - 1) / (targetWc * Tlag)^2);
Gc = (Tlag * s + 1) / (beta * Tlag * s + 1);
L = minreal(Gc * G0);
closedLoop = feedback(L, 1);

[gainMargin, phaseMargin, phaseCrossover, gainCrossover] = margin(L);
gainMarginDb = 20 * log10(gainMargin);
KvCompensated = Kv * dcgain(Gc);

assert(abs(KvCompensated - 10) < 1e-8, 'Topic 2 RED/GREEN failed: Kv is not 10 s^-1.');
assert(phaseMargin >= 50, 'Topic 2 RED/GREEN failed: phase margin is below 50 deg.');
```

```
assert(gainMarginDb >= 10, 'Topic 2 RED/GREEN failed: gain margin is below 10 dB.');

configure_report_graphics();
exportDpi = 300;
bodePlotSize = [100, 100, 1400, 850];
plotSize = [100, 100, 1300, 780];
tallPlotSize = [100, 100, 1300, 860];

fig = figure('Color', 'w', 'Units', 'pixels', 'Position', bodePlotSize);
bodeFrequency = logspace(-2, 2, 1200);
[magBefore, phaseBefore] = bode(G0, bodeFrequency);
[magAfter, phaseAfter] = bode(L, bodeFrequency);
magBeforeDb = 20 * log10(squeeze(magBefore));
magAfterDb = 20 * log10(squeeze(magAfter));
phaseBeforeDeg = squeeze(phaseBefore);
phaseAfterDeg = squeeze(phaseAfter);

subplot(2, 1, 1);
semilogx(bodeFrequency, magBeforeDb, 'Color', [0.00, 0.22, 0.80], 'LineWidth', 2.4); hold on;
semilogx(bodeFrequency, magAfterDb, 'Color', [0.85, 0.12, 0.00], 'LineWidth', 2.4);
yline(0, 'k--', 'LineWidth', 1.8);
grid on;
legend('Before compensation', 'After compensation', 'Location', 'southwest');
ylabel('Magnitude (dB)');
title('Topic 2 Bode plots');
subplot(2, 1, 2);
semilogx(bodeFrequency, phaseBeforeDeg, 'Color', [0.00, 0.22, 0.80], 'LineWidth', 2.4); hold on;
semilogx(bodeFrequency, phaseAfterDeg, 'Color', [0.85, 0.12, 0.00], 'LineWidth', 2.4);
yline(-180, 'k--', 'LineWidth', 1.8);
grid on;
xlabel('Frequency (rad/s)');
ylabel('Phase (deg)');
export_report_figure(fig, fullfile(figDir, 'bode_before_after.png'), exportDpi);
close(fig);

fig = figure('Color', 'w', 'Units', 'pixels', 'Position', bodePlotSize);
subplot(2, 1, 1);
semilogx(bodeFrequency, magAfterDb, 'Color', [0.00, 0.22, 0.80], 'LineWidth', 2.4); hold on;
yline(0, 'k--', 'LineWidth', 1.8);
if isfinite(gainCrossover) && gainCrossover > 0
    xline(gainCrossover, 'Color', [0.75, 0.00, 0.00], 'LineStyle', '--', 'LineWidth', 1.9);
end
if isfinite(phaseCrossover) && phaseCrossover > 0
    xline(phaseCrossover, 'Color', [0.10, 0.45, 0.10], 'LineStyle', ':', 'LineWidth', 1.9);
end
grid on;
ylabel('Magnitude (dB)');
title(sprintf('Compensated margins: PM = %.2f deg, GM = %.2f dB', phaseMargin, gainMarginDb));
subplot(2, 1, 2);
semilogx(bodeFrequency, phaseAfterDeg, 'Color', [0.00, 0.22, 0.80], 'LineWidth', 2.4); hold on;
yline(-180, 'k--', 'LineWidth', 1.8);
if isfinite(gainCrossover) && gainCrossover > 0
    xline(gainCrossover, 'Color', [0.75, 0.00, 0.00], 'LineStyle', '--', 'LineWidth', 1.9);
end
if isfinite(phaseCrossover) && phaseCrossover > 0
    xline(phaseCrossover, 'Color', [0.10, 0.45, 0.10], 'LineStyle', ':', 'LineWidth', 1.9);
end
grid on;
xlabel('Frequency (rad/s)');
```

```
ylabel('Phase (deg)');
export_report_figure(fig, fullfile(figDir, 'compensated_margin.png'), exportDpi);
close(fig);

t = linspace(0, 20, 2001);
fig = figure('Color', 'w', 'Units', 'pixels', 'Position', plotSize);
[stepOutput, stepTime] = step(closedLoop, t);
plot(stepTime, squeeze(stepOutput), 'Color', [0.00, 0.22, 0.80], 'LineWidth', 2.4);
grid on;
title('Closed-loop unit-step response');
xlabel('Time (s)');
ylabel('Output');
export_report_figure(fig, fullfile(figDir, 'closed_loop_step.png'), exportDpi);
close(fig);

theoreticalRampError = 1 / KvCompensated;
tRamp = linspace(0, 4000, 4001);
ramp = tRamp(:);
yRamp = lsim(closedLoop, ramp, tRamp);
eRamp = ramp - yRamp;
steadyRampError = mean(eRamp(end-200:end));
assert(abs(steadyRampError - theoreticalRampError) < 0.02, ...
    'Topic 2 RED/GREEN failed: long-time ramp error does not approach 1/Kv.');

fig = figure('Color', 'w', 'Units', 'pixels', 'Position', tallPlotSize);
subplot(2, 1, 1);
plot(tRamp, ramp, 'k--', 'LineWidth', 2.0); hold on;
plot(tRamp, yRamp, 'Color', [0.00, 0.22, 0.80], 'LineWidth', 2.4);
grid on;
legend('Ramp input', 'Output', 'Location', 'northwest');
ylabel('Signal');
title('Long-time ramp tracking verification');
subplot(2, 1, 2);
plot(tRamp, eRamp, 'Color', [0.85, 0.12, 0.00], 'LineWidth', 2.3); hold on;
yline(theoreticalRampError, 'k--', '1/K_v', 'LineWidth', 1.9);
grid on;
xlabel('Time (s)');
ylabel('Error');
title('Ramp tracking error converges to the velocity-error constant prediction');
export_report_figure(fig, fullfile(figDir, 'ramp_tracking.png'), exportDpi);
close(fig);

create_topic2_simulink_model(projectRoot, Tlag, beta * Tlag);

metricsPath = fullfile(buildDir, 'topic2_metrics.txt');
fid = fopen(metricsPath, 'w', 'n', 'UTF-8');
fprintf(fid, 'K = %.6f\n', K);
fprintf(fid, 'Kv = %.6f s^-1\n', KvCompensated);
fprintf(fid, 'Gc(s) = (%.6f s + 1) / (%.6f s + 1)\n', Tlag, beta * Tlag);
fprintf(fid, 'beta = %.6f\n', beta);
fprintf(fid, 'T = %.6f s\n', Tlag);
fprintf(fid, 'phase_margin_deg = %.6f\n', phaseMargin);
fprintf(fid, 'gain_margin_db = %.6f\n', gainMarginDb);
fprintf(fid, 'gain_crossover_rad_per_s = %.6f\n', gainCrossover);
fprintf(fid, 'theoretical_ramp_error = %.6f\n', theoreticalRampError);
fprintf(fid, 'steady_ramp_error = %.6f\n', steadyRampError);
fprintf(fid, 'ramp_simulation_time_s = %.6f\n', tRamp(end));
fclose(fid);
```

```
fprintf('Topic 2 GREEN: Kv=%.4f s^-1, PM=%.4f deg, GM=%.4f dB, ramp error=%.4f (theory %.4f)\n', ...
    KvCompensated, phaseMargin, gainMarginDb, steadyRampError, theoreticalRampError);

function create_topic2_simulink_model(projectRoot, Tlag, lagDenCoeff)
    modelName = 'topic2_compensated_feedback';
    modelPath = fullfile(projectRoot, 'matlab', [modelName '.slx']);
    screenshotPath = fullfile(projectRoot, 'figures', 'topic2', 'simulation_structure.png');

    if bdIsLoaded(modelName)
        close_system(modelName, 0);
    end
    if exist(modelPath, 'file')
        load_system(modelPath);
        set_param(modelName, 'StopTime', '20', 'ScreenColor', 'white');
        assert_required_blocks(modelName, {'Gc', 'G0', 'Unity Feedback'});
        set_param([modelName '/Gc'], ...
            'Numerator', sprintf('[%.12g 1]', Tlag), ...
            'Denominator', sprintf('[%.12g 1]', lagDenCoeff));
        set_param([modelName '/G0'], 'Numerator', '40', 'Denominator', '[1 5 4 0]');
        set_param([modelName '/Unity Feedback'], 'Gain', '1');
        apply_topic2_visual_layout(modelName);
        style_simulink_model(modelName);
        assert_orthogonal_lines(modelName);
        set_param(modelName, 'ZoomFactor', 'FitSystem');
        save_system(modelName, modelPath);
        export_simulink_image(modelName, screenshotPath);
        close_system(modelName, 0);
        return;
    end
    new_system(modelName);
    open_system(modelName);
    set_param(modelName, 'Location', [80, 80, 1580, 680], 'StopTime', '20', 'ScreenColor', 'white');

    add_block('simulink/Sources/Step', [modelName '/Step'], 'Position', [40, 130, 130, 210]);
   add_block('simulink/Math Operations/Sum', [modelName '/Sum'], 'Inputs', '+-', 'IconShape', 'round',
'Position', [180, 145, 210, 195]);
    add_block('simulink/Continuous/Transfer Fcn', [modelName '/Gc'], ...
        'Numerator', sprintf('[%.12g 1]', Tlag), ...
        'Denominator', sprintf('[%.12g 1]', lagDenCoeff), ...
        'Position', [290, 130, 470, 210]);
    add_block('simulink/Continuous/Transfer Fcn', [modelName '/G0'], ...
        'Numerator', '40', 'Denominator', '[1 5 4 0]', ...
        'Position', [560, 130, 740, 210]);
    add_block('simulink/Math Operations/Gain', [modelName '/Unity Feedback'], ...
        'Gain', '1', 'Position', [560, 370, 740, 430]);
    add_block('simulink/Sinks/Scope', [modelName '/Scope'], 'Position', [870, 120, 950, 210]);
    add_block('simulink/Sinks/Out1', [modelName '/c'], 'Position', [1010, 155, 1045, 185]);

    add_line(modelName, 'Step/1', 'Sum/1', 'autorouting', 'on');
    add_line(modelName, 'Sum/1', 'Gc/1', 'autorouting', 'on');
    add_line(modelName, 'Gc/1', 'G0/1', 'autorouting', 'on');
    add_line(modelName, 'G0/1', 'Scope/1', 'autorouting', 'on');
    add_line(modelName, 'G0/1', 'c/1', 'autorouting', 'on');
    add_line(modelName, 'G0/1', 'Unity Feedback/1', 'autorouting', 'on');
    add_line(modelName, 'Unity Feedback/1', 'Sum/2', 'autorouting', 'on');

    apply_topic2_visual_layout(modelName);
    style_simulink_model(modelName);
    assert_orthogonal_lines(modelName);
```

```
    set_param(modelName, 'ZoomFactor', 'FitSystem');
    save_system(modelName, modelPath);
    export_simulink_image(modelName, screenshotPath);
    close_system(modelName, 0);
end

function export_simulink_image(modelName, screenshotPath)
    try
        Simulink.BlockDiagram.exportToImage(modelName, screenshotPath);
    catch
        print(['-s' modelName], '-dpng', '-r300', screenshotPath);
    end
    annotate_simulink_icons(modelName, screenshotPath);
    assert_white_image_background(screenshotPath);
end

function annotate_simulink_icons(modelName, screenshotPath)
    rgb = imread(screenshotPath);
    if ndims(rgb) == 2
        rgb = repmat(rgb, 1, 1, 3);
    elseif size(rgb, 3) > 3
        rgb = rgb(:, :, 1:3);
    end
    [height, width, ~] = size(rgb);
    switch modelName
        case 'topic2_compensated_feedback'
            rgb = draw_step_icon(rgb, [0.040, 0.240, 0.055, 0.080], width, height);
            rgb = draw_scope_icon(rgb, [0.925, 0.222, 0.050, 0.070], width, height);
            rgb = draw_gain_one_icon(rgb, [0.405, 0.820, 0.070, 0.070], width, height);
    end
    imwrite(rgb, screenshotPath);
end

function rgb = draw_text_icon(rgb, label, box, width, height)
    pos = rel_box(box, width, height);
    rgb = insertText(rgb, [pos(1) + pos(3) / 2, pos(2) + pos(4) / 2], label, ...
        'AnchorPoint', 'Center', 'FontSize', max(22, round(pos(4) * 0.28)), ...
        'TextColor', 'black', 'BoxOpacity', 0);
end

function rgb = draw_gain_one_icon(rgb, box, width, height)
    pos = rel_box(box, width, height);
    rgb = insertText(rgb, [pos(1) + pos(3) / 2, pos(2) + pos(4) / 2], '1', ...
        'AnchorPoint', 'Center', 'FontSize', max(34, round(pos(4) * 0.42)), ...
        'TextColor', 'black', 'BoxOpacity', 0);
end

function rgb = draw_step_icon(rgb, box, width, height)
    pos = rel_box(box, width, height);
   x = [pos(1) + 0.18 * pos(3), pos(1) + 0.45 * pos(3), pos(1) + 0.45 * pos(3), pos(1) + 0.82 * pos(3)];
   y = [pos(2) + 0.68 * pos(4), pos(2) + 0.68 * pos(4), pos(2) + 0.32 * pos(4), pos(2) + 0.32 * pos(4)];
     rgb = insertShape(rgb, 'Line', [x(1), y(1), x(2), y(2); x(2), y(2), x(3), y(3); x(3), y(3),
x(4), y(4)], ...
        'Color', 'black', 'LineWidth', 3);
end

function rgb = draw_scope_icon(rgb, box, width, height)
    pos = rel_box(box, width, height);
    rgb = insertShape(rgb, 'Rectangle', [pos(1) + 0.18 * pos(3), pos(2) + 0.20 * pos(4), 0.64 * pos(3),
```

```
0.58 * pos(4)], ...
        'Color', 'black', 'LineWidth', 3);
    for k = 1:2
        x = pos(1) + (0.18 + 0.64 * k / 3) * pos(3);
       rgb = insertShape(rgb, 'Line', [x, pos(2) + 0.20 * pos(4), x, pos(2) + 0.78 * pos(4)], 'Color',
[80, 80, 80], 'LineWidth', 1);
    end
    y = pos(2) + 0.49 * pos(4);
     rgb = insertShape(rgb, 'Line', [pos(1) + 0.18 * pos(3), y, pos(1) + 0.82 * pos(3), y], 'Color',
[80, 80, 80], 'LineWidth', 1);
end

function pos = rel_box(box, width, height)
    pos = [box(1) * width, box(2) * height, box(3) * width, box(4) * height];
end

function apply_topic2_visual_layout(modelName)
    set_param([modelName '/Step'], 'Position', [40, 130, 130, 210]);
    set_param([modelName '/Scope'], 'Position', [870, 120, 950, 210]);
    force_icon_display([modelName '/Step']);
    force_icon_display([modelName '/Scope']);
    force_icon_display([modelName '/Unity Feedback']);
end

function force_icon_display(blockPath)
    params = get_param(blockPath, 'ObjectParameters');
    if isfield(params, 'IconDisplay')
        try
            set_param(blockPath, 'IconDisplay', 'Graphic');
        catch
        end
    end
    if isfield(params, 'ShowName')
        set_param(blockPath, 'ShowName', 'on');
    end
end

function configure_report_graphics()
    set(0, 'DefaultFigureVisible', 'off');
    set(0, 'DefaultFigureColor', 'w');
    set(0, 'DefaultAxesColor', 'w');
    set(0, 'DefaultAxesXColor', 'k');
    set(0, 'DefaultAxesYColor', 'k');
    set(0, 'DefaultAxesZColor', 'k');
    set(0, 'DefaultAxesFontName', 'Arial');
    set(0, 'DefaultAxesFontSize', 12);
    set(0, 'DefaultAxesLineWidth', 1.2);
    set(0, 'DefaultTextColor', 'k');
    set(0, 'DefaultTextFontName', 'Arial');
    set(0, 'DefaultTextFontSize', 12);
    set(0, 'DefaultLineLineWidth', 2.2);
end

function export_report_figure(fig, outputPath, exportDpi)
    style_report_figure(fig);
   exportgraphics(fig, outputPath, 'Resolution', exportDpi, 'BackgroundColor', 'white', 'ContentType',
'image');
    assert_white_image_background(outputPath);
end
```

```
function style_report_figure(fig)
    set(fig, 'Color', 'w', 'InvertHardcopy', 'off');
    axesHandles = findall(fig, 'Type', 'axes');
    for idx = 1:numel(axesHandles)
        set(axesHandles(idx), 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'ZColor', 'k', ...
            'FontName', 'Arial', 'FontSize', 12, 'LineWidth', 1.2, ...
            'GridColor', [0.25, 0.25, 0.25], 'MinorGridColor', [0.55, 0.55, 0.55]);
    end
    textHandles = findall(fig, 'Type', 'text');
    if ~isempty(textHandles)
        set(textHandles, 'Color', 'k', 'FontName', 'Arial');
    end
    lineHandles = findall(fig, 'Type', 'line');
    if ~isempty(lineHandles)
        set(lineHandles, 'LineWidth', 2.2);
    end
    constantLines = findall(fig, 'Type', 'ConstantLine');
    if ~isempty(constantLines)
        set(constantLines, 'LineWidth', 1.9, 'Color', 'k');
    end
    legends = findall(fig, 'Type', 'legend');
    if ~isempty(legends)
       set(legends, 'Color', 'w', 'TextColor', 'k', 'EdgeColor', 'k', 'FontName', 'Arial', 'FontSize',
11);
    end
end

function style_simulink_model(modelName)
    set_param(modelName, 'ScreenColor', 'white');
    blockHandles = find_system(modelName, 'FindAll', 'on', 'Type', 'block');
    for idx = 1:numel(blockHandles)
        params = get_param(blockHandles(idx), 'ObjectParameters');
        if isfield(params, 'ForegroundColor')
            set_param(blockHandles(idx), 'ForegroundColor', 'black');
        end
        if isfield(params, 'BackgroundColor')
            set_param(blockHandles(idx), 'BackgroundColor', 'white');
        end
        if isfield(params, 'FontSize')
            set_param(blockHandles(idx), 'FontSize', '14');
        end
    end
end

function assert_required_blocks(modelName, blockNames)
    for idx = 1:numel(blockNames)
        blockPath = [modelName '/' blockNames{idx}];
        assert(getSimulinkBlockHandle(blockPath) ~= -1, ['Required block missing: ' blockPath]);
    end
end

function postprocess_simulink_image(imagePath)
    rgb = imread(imagePath);
    if ndims(rgb) == 2
        rgb = repmat(rgb, 1, 1, 3);
    elseif size(rgb, 3) > 3
        rgb = rgb(:, :, 1:3);
    end
```

```
    if ~isa(rgb, 'uint8')
        rgb = uint8(255 * double(rgb) / double(intmax(class(rgb))));
    end
    gray = 0.299 * double(rgb(:, :, 1)) + 0.587 * double(rgb(:, :, 2)) + 0.114 * double(rgb(:, :, 3));
    cornerSamples = [gray(1:20, 1:20); gray(1:20, end-19:end); gray(end-19:end, 1:20); gray(end-19:end,
end-19:end)];
    output = uint8(255 * ones(size(rgb)));
    if median(cornerSamples(:)) < 80
        contentMask = gray > 100;
    else
        contentMask = gray < 190;
    end
    thickMask = conv2(double(contentMask), ones(5), 'same') > 0;
    for channelIdx = 1:3
        channel = output(:, :, channelIdx);
        channel(thickMask) = 0;
        output(:, :, channelIdx) = channel;
    end
    imwrite(output, imagePath);
end

function assert_white_image_background(imagePath)
    rgb = imread(imagePath);
    if ndims(rgb) == 2
        rgb = repmat(rgb, 1, 1, 3);
    elseif size(rgb, 3) > 3
        rgb = rgb(:, :, 1:3);
    end
    gray = 0.299 * double(rgb(:, :, 1)) + 0.587 * double(rgb(:, :, 2)) + 0.114 * double(rgb(:, :, 3));
    cornerSamples = [gray(1:20, 1:20); gray(1:20, end-19:end); gray(end-19:end, 1:20); gray(end-19:end,
end-19:end)];
    assert(mean(cornerSamples(:)) > 245, 'Exported image background is not white.');
end

function assert_orthogonal_lines(modelName)
    lineHandles = find_system(modelName, 'FindAll', 'on', 'Type', 'line');
    for idx = 1:numel(lineHandles)
        points = get_param(lineHandles(idx), 'Points');
        for pointIdx = 1:(size(points, 1) - 1)
            dx = abs(points(pointIdx + 1, 1) - points(pointIdx, 1));
            dy = abs(points(pointIdx + 1, 2) - points(pointIdx, 2));
            assert(dx == 0 || dy == 0, 'Simulink line is not routed with right angles.');
        end
    end
end
```

0.10. 结论

本文按 Bode 图法完成了串联滞后校正装置设计。所得到的校正装置为 $G_{c(s)}=\frac{20s+1}{356.773145s+1}$。MATLAB 验证表明，校正后系统满足 $K_v=10\text{s}^{-1}$、相位裕度不小于 50°、增益裕度不小于 10 dB 的设计要求。